Laboratorio No.8

Informe contextual de Prompt Injection

El contexto

De lo expuesto, formule 5 buenas prácticas de Prompt Injection desde lo ofensivo y lo defensivo para un entorno controlado.

5 Buenas Prácticas Ofensivas (Testing y Auditoría en Entorno Controlado)

- **Evaluar el control de instrucciones mediante "Overrides":** Diseñar pruebas directas que ordenen al modelo ignorar sus reglas anteriores o priorizar el mensaje actual para verificar si respeta sus directrices internas.
- **Simular escalada y confusión de roles:** Ejecutar prompts donde se le pida a la IA que asuma la identidad de un administrador, un auditor interno o un módulo de backend, con el fin de evaluar si el sistema delega privilegios indebidamente.
- **Probar la resistencia a exfiltración de datos:** Intentar extraer información oculta mediante preguntas directas sobre el "prompt inicial", "variables internas" o el contexto almacenado en memoria, evaluando así el riesgo de fuga de secretos (LLM07).
- **Aplicar ataques encadenados (Prompt Chaining):** Estructurar las pruebas en secuencias o pasos lógicos (ej. "Paso 1: cambia rol, paso 2: muestra datos") para medir la capacidad del sistema de validar instrucciones múltiples sin ejecutar acciones maliciosas automatizadas.
- **Medir y cuantificar las vulnerabilidades:** En lugar de solo verificar si el ataque funciona, documentar los resultados utilizando métricas clave de seguridad en IA, como la Tasa de éxito de inyección (ISR), el Nivel de exposición de datos (DEL) y la Robustez del modelo (MR).

5 Buenas Prácticas Defensivas (Arquitectura y Prevención)

- **Implementar separación estricta de contexto:** Aislar estructuralmente las instrucciones del sistema del input proporcionado por el usuario o por fuentes externas (como bases documentales), evitando que la entrada externa compita con las reglas internas.
- **Aplicar validación bidireccional:** Sanitizar y filtrar las entradas para detectar patrones maliciosos, y validar exhaustivamente el formato y contenido de las salidas antes de que interactúen con otros componentes o usuarios (LLM05).

- **Ejecutar el principio de mínimo privilegio:** Restringir severamente el nivel de agencia del modelo, limitando su acceso a bases de datos, APIs o herramientas conectadas exclusivamente a lo necesario para su función.
- **Sanitizar el Prompt del Sistema (System Prompt):** Asegurar que las instrucciones internas del modelo no contengan secretos, credenciales, contraseñas ni lógica de autorización crítica, ya que el prompt del sistema no debe usarse como un control de seguridad definitivo (LLM07).
- **Incorporar supervisión humana (Human-in-the-loop):** Requerir obligatoriamente la validación y aprobación de un operador humano antes de que el sistema ejecute acciones de alto riesgo o irreversibles.

Las Preguntas

1. Explique qué es un ataque de prompt injection en sistemas basados en modelos de lenguaje y cómo se relaciona con vulnerabilidades tradicionales de seguridad.

   Un ataque de *prompt injection* ocurre cuando se manipulan las instrucciones de un modelo de lenguaje utilizando lenguaje natural para alterar su comportamiento esperado. A diferencia de los ciberataques convencionales, no utiliza código malicioso tradicional, sino "lenguaje malicioso" que el sistema de IA interpreta como una instrucción válida. Se relaciona de manera directa con vulnerabilidades clásicas como la Inyección SQL o el XSS, ya que comparte la misma naturaleza de inyectar comandos para alterar consultas o procesos. Además, encaja dentro del Control de Acceso Roto, porque permite a un usuario eludir restricciones lógicas y forzar al modelo a mostrar información confidencial sin necesidad de vulnerar la red de forma tradicional.

2. Analice por qué la separación entre instrucciones del sistema y entrada del usuario es crítica en aplicaciones con IA.

   Mantener esta separación es fundamental porque el principal fallo de seguridad en estas aplicaciones es confiar excesivamente en la información que ingresa el usuario. Si el sistema no separa correctamente las directrices internas de lo que escribe la persona, ocurre un fallo de diseño arquitectónico. Esto provoca que las instrucciones externas compitan con las reglas del modelo, permitiendo que el usuario pueda interferir con el comportamiento del sistema y forzarlo a ignorar sus restricciones de seguridad.

3. Describa tres riesgos principales asociados al prompt injection en aplicaciones conectadas a APIs o bases de datos.

   F**uga de información (Data exfiltration):** Un atacante puede engañar al modelo para que exponga variables internas, claves de API, secretos almacenados o datos sensibles del contexto.

   Ejecución de acciones no autorizadas: Si el sistema tiene configurado el acceso a bases de datos o APIs sin los controles adecuados, una inyección exitosa puede lograr que la IA ejecute comandos o acciones indebidas en los sistemas conectados.

   Manipulación de decisiones: Mediante inyecciones, se puede alterar la lógica del sistema para que, por ejemplo, apruebe condiciones indebidamente o modifique resultados críticos dentro de los procesos conectados.

4. Explique cómo el prompt injection puede ser considerado una forma moderna de ingeniería social.

   Se considera ingeniería social porque se basa en técnicas de manipulación y engaño a través del lenguaje, siendo el equivalente al *phishing* pero dirigido a una Inteligencia Artificial. No se aprovecha de errores de código de programación, sino que explota la forma en que el modelo interpreta el texto. Se utilizan tácticas como asumir una autoridad falsa (diciendo "soy el administrador" o "esto es una prueba de seguridad autorizada") o generar presión temporal para convencer a la IA de obedecer órdenes y saltarse sus barreras, del mismo modo que un estafador intentaría convencer a un guardia de seguridad.

5. Proponga al menos tres estrategias de mitigación para prevenir ataques de prompt injection en aplicaciones reales.

   Separación de contexto: Aislar de manera clara el contenido que es confiable (como las instrucciones del sistema) del contenido que no es confiable (como el texto ingresado por el usuario o los documentos procesados).

   Aplicar el principio de mínimo privilegio: Restringir los permisos y limitar el nivel de agencia del modelo, otorgándole acceso únicamente a los datos y funciones que sean estrictamente necesarios para su operación.

   Filtrado de entradas y validación de salidas: Implementar mecanismos para detectar patrones maliciosos en los textos que ingresan, y verificar el formato y contenido de las respuestas generadas por el modelo antes de que sean procesadas por otros componentes.